Advanced Computer-Aided VLSI System Design
Lab3: Insert Scan Chain

TA: Can-Lin Wu r13943017@ntu.edu.tw

Objectives

In this lab, you will learn how to insert scan chain into a synthesized gate level design.

Data Preparation

1. Create a work directory and copy the lab files into it.
2. Check if you have these files:

Filename	Description
ALU_syn.v	Synthesized gate level Verilog netlist for the simple ALU (no scan chain yet)
.synopsys_dc.setup	Synopsys Design Compiler setup file, which defines search paths, library name, etc.
constraints.tcl	The constraints that are used in the synthesis lab.
dft.tcl	Reference script in this lab.

3. Check the contents of these files.

Note: If you have the file " **synopsys_dc.setup** " instead of " **.synopsys_dc.setup** " (there's a dot in the beginning), run the following command to change the file name. Because Design Compiler can only read the file " **.synopsys_dc.setup** " which has a dot in the beginnig.

% **mv synopsys_dc.setup .synopsys_dc.setup**

Environment Setup

% **cb**

Invoke DftCompiler

Dft Compiler is actually embedded in the Design Compiler. To invoke Dft Compiler, you can do either one

% **dc_shell** (command mode)

% **dv &** (GUI mode)

I encourage everybody to use command mode because:

a. Command mode helps you to keep a record of what you have done.
b. Command mode runs more efficiently than GUI mode.
c. Command mode helps you to lookup the manual/reference quickly.

In spite of the above advantages, command mode sometimes is not as good as GUI mode in terms of debugging the schematic problem. We will use command mode throughout this Lab. You are welcome to try the GUI mode by yourself.

NOTE: maybe you see some error message like "Error: current design not defined." just ignore it for now.

STEP 1: Read Input Files

1. Please check there is no error message when starting the "dc_shell". If there are errors in the windows, please check the .synopsys_dc.setup. Type either one of these lines to read your gate level netlist (The circuit after synthesis).

dc_shell > **read_verilog ALU_syn.v**
dc_shell > **read_file ALU_syn.v -format verilog**

2. Set the working design to you top design. In this case, set ALU as the working design.
   dc_shell > **current_design ALU**
3. Resolve the design references and check if there is any errors.
   dc_shell > **link**
   dc_shell > **check_design**
4. Set the design constraints and check if the designs have any violations. The constraints.tcl is based on the constraints that you used in the synthesis lab.
   dc_shell > **source constraints.tcl**
   dc_shell > **report_constraint -all_violators**
5. To obtain a timing/area/power report of your original design, type (where ALU is your top design)
   dc_shell > **report_area > ALU_syn.area_rpt**
   dc_shell > **report_timing > ALU_syn.timing_rpt**
   dc_shell > **report_power > ALU_syn.power_rpt**

STEP 2: Select scan style

Define the default scan style for the insert_dft command if a scan style is not specified with the set_scan_style command. This variable must identify one of the following supported scan styles: multiplexed_flip_flop, clocked_scan, lssd, aux_clock_lssd, combinational, or none. You can skip this step because the default is multiplexed_flip_flop.

dc_shell > **set test_default_scan_style multiplexed_flip_flop**

STEP 3: Set ATE configuration and create test protocol

The timing of the test clock is based on the test_default_delay, test_default_bidir_delay, test_default_strobe, and test_default_period variables.

dc_shell > **set test_default_delay 0**
dc_shell > **set test_default_bidir_delay 0**
dc_shell > **set test_default_strobe 40**
dc_shell > **set test_default_period 100**

To create a test protocol for a non-scan design, you can just type
dc_shell > **create_test_protocol -infer_asynch -infer_clock**

When -infer_asynch is specified, create_test_protocol infers asynchronous set and reset signals in the design, and places them at off state during scan shifting. When -infer_clock is specified, create_test_protocol infers test clock pins from the design, and pulses them during scan shifting.

(Optional) In this lab, DFT compiler can identify your clock and asynchronous reset automatically. Instead of automatic identification, you can also specify these signals by the set_dft_signal command. For example:

```
set_dft_signal -view existing_dft -type ScanClock -port clk \
               -timing [list 45 55]
set_dft_signal -view existing_dft -type reset -port reset \
```

-active_state 0
create_test_protocol

STEP 4: Pre-scan Check

Check if there is any design constraint violations before scan insertion.
dc_shell > **report_constraint -all_violators**

Perform pre-scan test design rule checking.
dc_shell > **dft_drc**

Question 1: How many scan cells do you have? _____
Question 2: How many rule violation do you have? _____
Note: If there were violations, you should stop to fix your code.

STEP 5: Scan specification

This step tells the Dft Compiler how many scan chains you want. You can also specify the names of scan related pins (scan_enable, scan_in, scan_out). We will let Dft Compiler to choose the pin names for us.
dc_shell > **set_scan_configuration -chain_count 1**

STEP 6: Scan preview

This step checks your scan specification for consistency. Please type
dc_shell > **preview_dft**

Question 1: How many scan chains will you have? _____
Question 2: What are their pin names? _____

STEP 7: scan chain synthesis

Stitch your scan cells into a chain. And do some more optimizations.
dc_shell > **insert_dft**

STEP8: Post-scan check

Check if there is any design constraint violations after scan insertion.
dc_shell > **report_constraint -all_violators**

Perform post-scan test design rule checking.
dc_shell > **dft_drc**

STEP 9: Reports

Report the scan cells and the scan paths
dc_shell > **report_scan_path -view existing -chain all > ALU_syn_dft.scan_path**
dc_shell > **report_scan_path -view existing -cell all > ALU_syn_dft.scan_cell**

To obtain a timing/area report of your scan_inserted design, type
dc_shell > **report_area > ALU_syn_dft.area_rpt**
dc_shell > **report_timing > ALU_syn_dft.timing_rpt**
dc_shell > **report_power > ALU_syn_dft.power_rpt**

Examine the report files of our scan_inserted design. Compare these reports with those of the non-scan design.

Question 1: How much is the total cell area overhead of scan? _____%

Question 2: How much is the timing overhead of scan? _____%

STEP 10: Write out files

Write scandef file for backend scan chain reordering

dc_shell > **write_scan_def -output ALU_syn_dft.scandef**

Write test protocol for later usage in the ATPG.

dc_shell > **write_test_protocol -output ALU_syn_dft.spf**

To output our scan-inserted netlist, type

dc_shell > **write -hierarchy -format verilog -output ALU_syn_dft.v**

dc_shell > **write -hierarchy -format ddc -output ALU_syn_dft.ddc**

The sdf (standard delay format) file is for timing analysis. Our next tool, Primetime, will need this file.

dc_shell > **write_sdf -version 2.1 ALU_syn_dft.sdf**

Questions/Comments: Check your Verilog netlist.

1. **What type of DFF before dft_compiler? _____**
2. **What type of DFF after dft_compiler? _____**
3. **Which two pins did dft_compiler add? _____ and _____**
4. **Where is scan output pin? _____**

dft_compiler share the scan_output pin with the functional output pin, so there is NO dedicated scan output pin. This is done for saving the number of pins. If you want dft_compiler to create a dedicate scan out pin, use this command before you insert the scan chain.

dc_shell > **set_scan_configuration -create_dedicated_scan_out_ports true**

Checkpoints

Please check with TAs before leaving this lab to make sure the following goals are accomplished and to get credits.

1. Show your DFT results without any DRC violations.

```
dc_shell> dft_drc
In mode: Internal_scan...
  Design has scan chains in this mode
  Design is scan routed
  Post-DFT DRC enabled

Information: Starting test design rule checking. (TEST-222)
  Loading test protocol
  ...basic checks...
  ...basic sequential cell checks...
  ...checking vector rules...
  ...checking clock rules...
  ...checking scan chain rules...
  ...checking scan compression rules...
  ...checking X-state rules...
  ...checking tristate rules...
  ...extracting scan details...

---------------------------------------------------------------
  DRC Report

  Total violations: 0

---------------------------------------------------------------


Test Design rule checking did not find violations

---------------------------------------------------------------
  Sequential Cell Report

  0 out of 28 sequential cells have violations

---------------------------------------------------------------

SEQUENTIAL CELLS WITHOUT VIOLATIONS
      *  28 cells are valid scan cells

Information: Test design rule checking completed. (TEST-123)
1
```

2. Answer the questions in this lab.